軟體工程課程:
軟體設計-模組設計

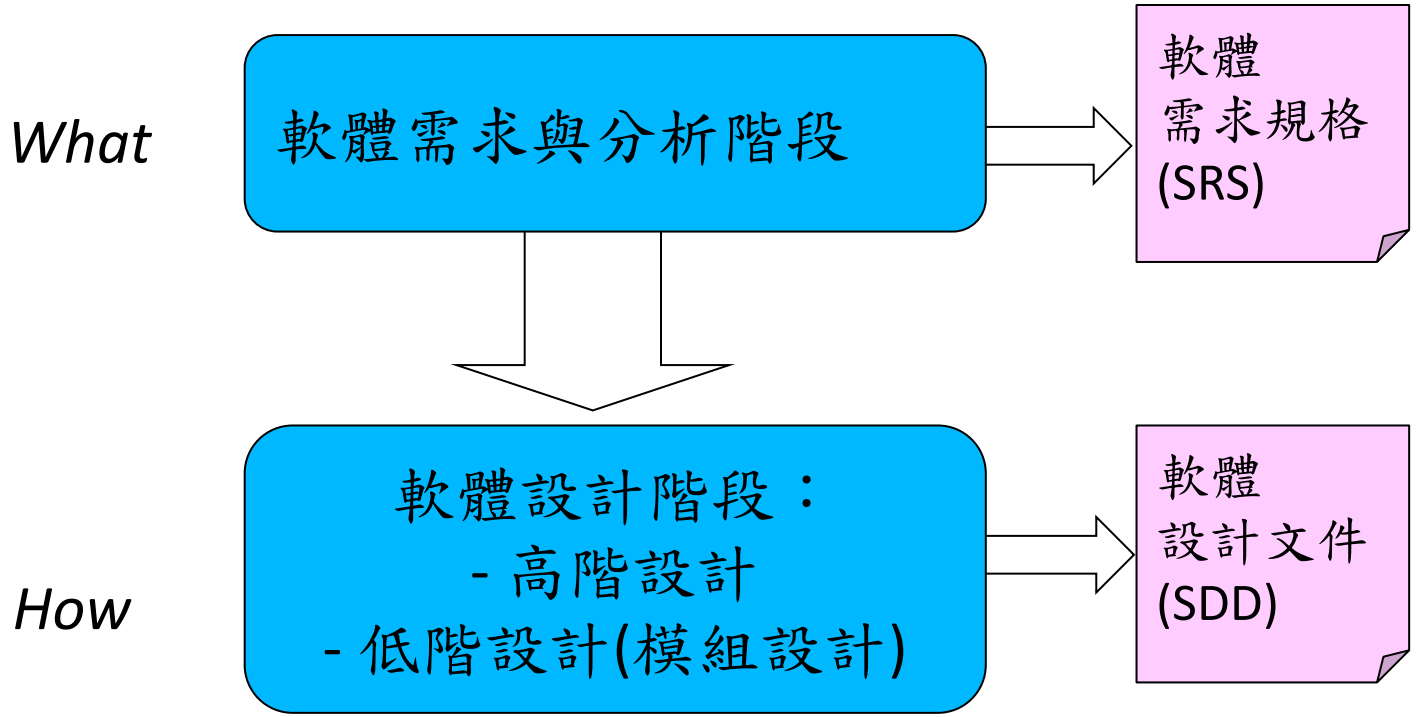
海大資工 馬尚彬

課程目的

- 瞭解如何透過UML進行模組結構設計(UML類別圖)
- 瞭解基礎的物件導向設計觀念
- 瞭解類別圖與元件圖的對應
- 瞭解類別圖之運用方式

教學影片連結：
https://www.youtube.com/playlist?list=PLoIEfC1NWpeLJOEeG35QBWz-15XzDYmVh

軟體設計概念


What
軟體需求與分析階段
軟體
需求規格
(SRS)
How
軟體設計階段：
- 高階設計
- 低階設計(模組設計)
軟體
設計文件
(SDD)

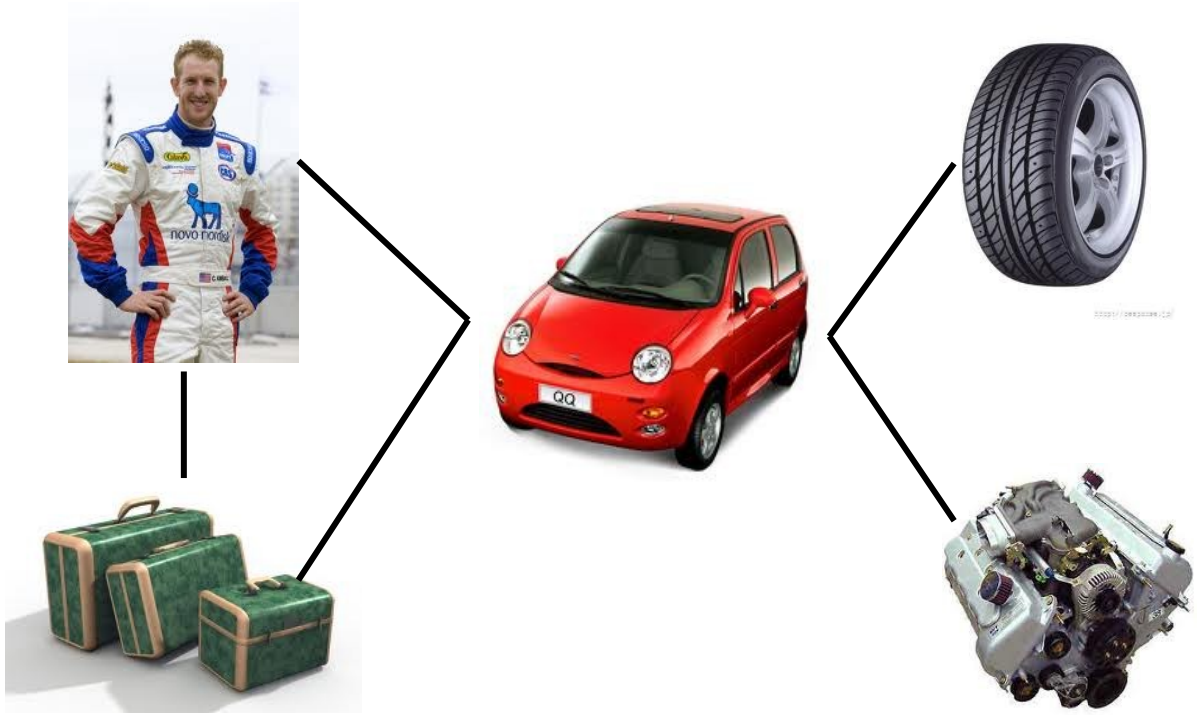
Class Diagram (類別圖)

- Class diagram 是用來描述系統中物件的類型，以及類型間的各種靜態關係。

- 基本的分析/設計步驟：
  - (1)從問題領域界定出可能的物件/類別。
  - (2)根據每個類別的特性，決定該類別的責任(Responsibility)。
  - (2)決定類別間的互動合作關係(Collaboration)。

物件(Object)

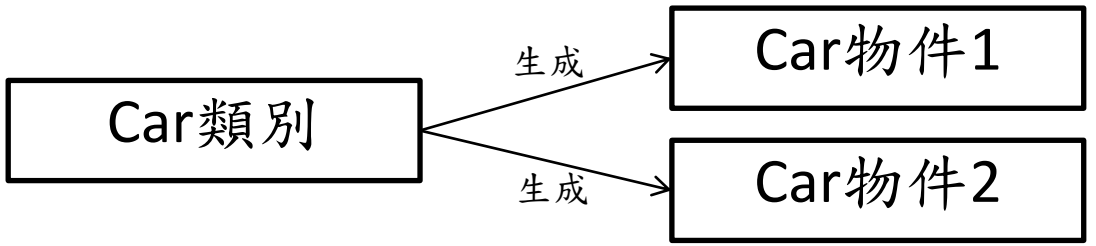
- 物件導向程式設計的主體是物件，他們合作完成一個系統的運作。
- 物件導向程式(object-oriented program)可視為多個具互動關係之物件的集合



類別(Class)與物件(Object)

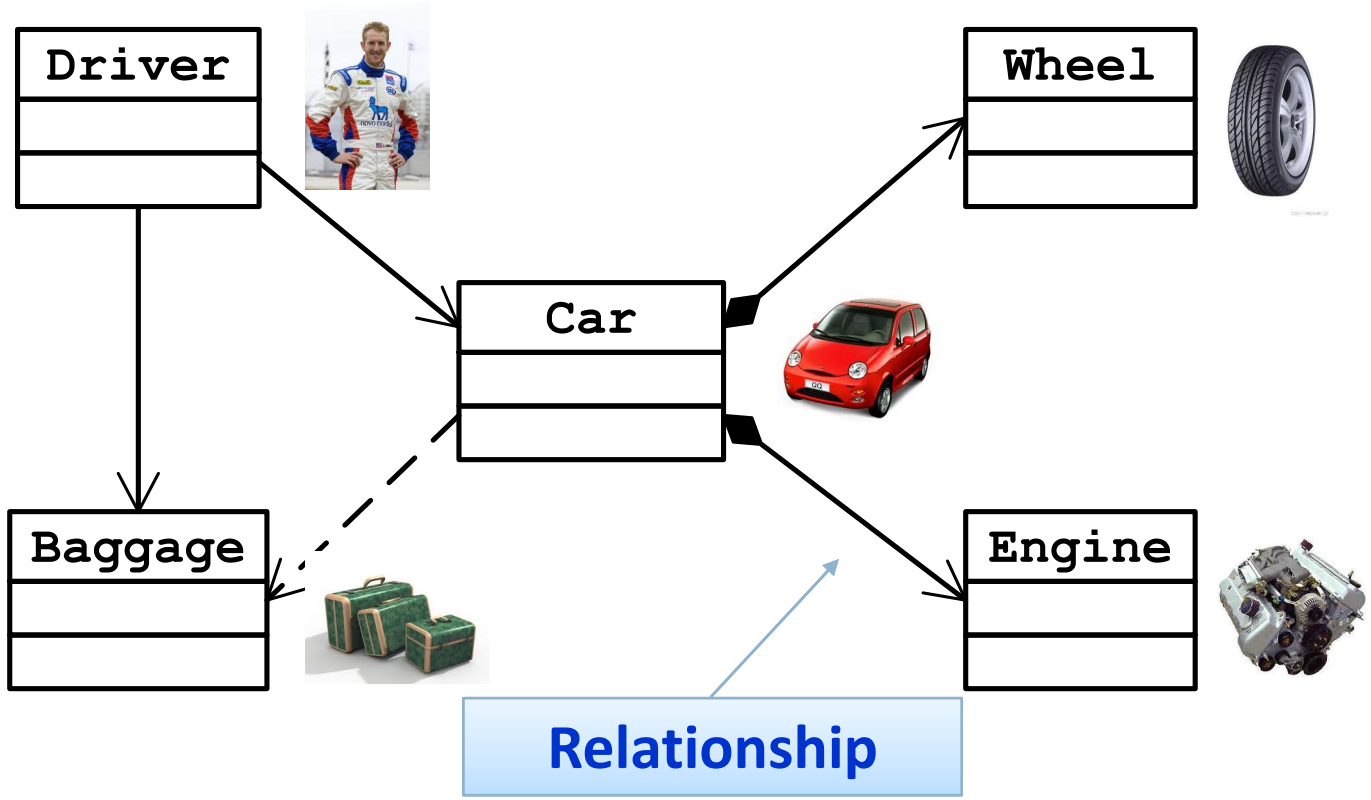
- 類別(class)
  - 類別是抽象的模板(template)，不具備實際的值(value)。
  - 一群具相同屬性(attribute)、操作(operation)或方法(method)、關聯(association)與行為(behavior)的物件之模版，描述具相同特徵的一群物件。

- 物件(object)
  - 又稱實例(instance)，是具象的實體。
  - 由類別產生，具有實際的值。



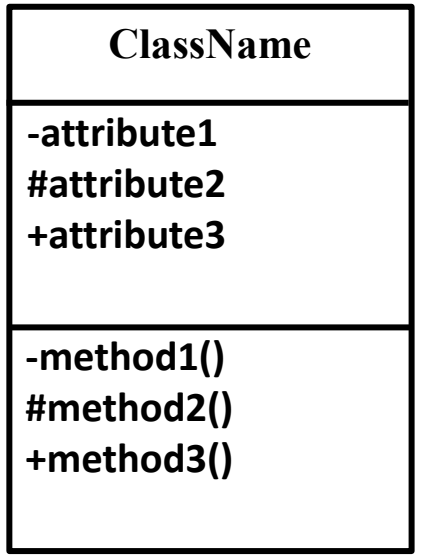
UML Class Diagram: 範例

- 統一塑模語言(UML, Unified Modeling Language)可用來表達物件內涵和物件間的互動關係

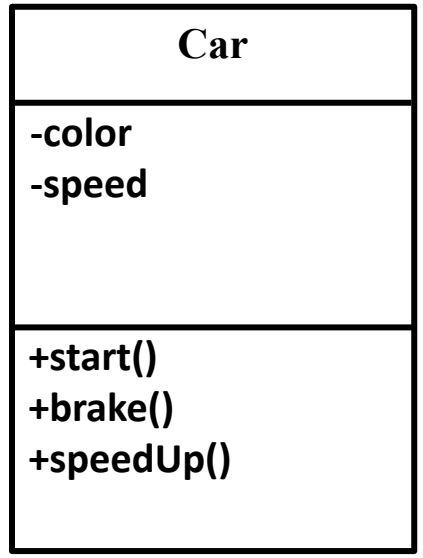


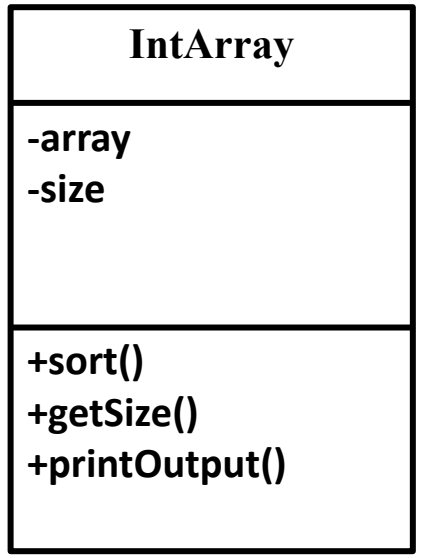
Class in UML

Class Diagram:



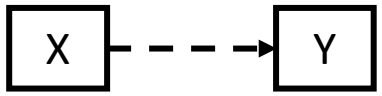
Example:

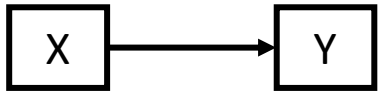




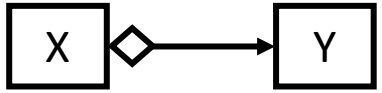

```
+    public
#    protected
-    private
```

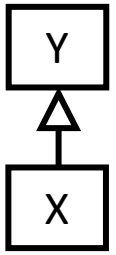
類別關聯性(Relationship)

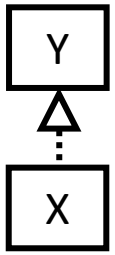
- Dependency 

(uses a)
- Association 

(knows a)
- Composition 

(has a)
- Aggregation 

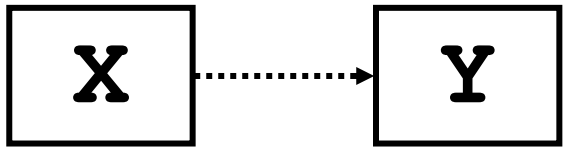
(has a)
- Inheritance 

(is a)
- Implementation 

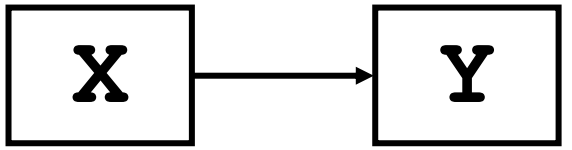
(can do)

“Uses a” ⇔ “Knows a” Relationship

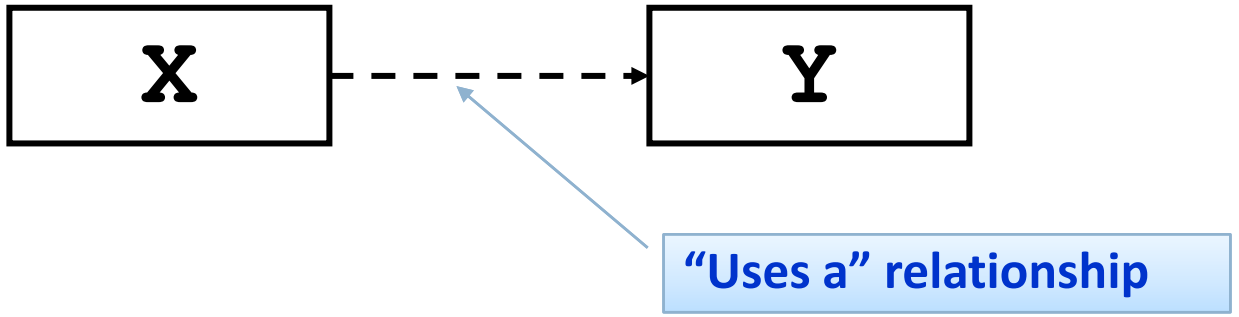
- “Uses a” (使用)
  - Dependency (相依)
  - X使用Y：X呼叫Y的method
  - **Short term relationship (短期關係)**



- “Knows a” (熟識)
  - Association (關聯)
  - X熟識Y：X包含Y的reference (Y是X的一個資料成員)
  - **Long term relationship (長期關係)**

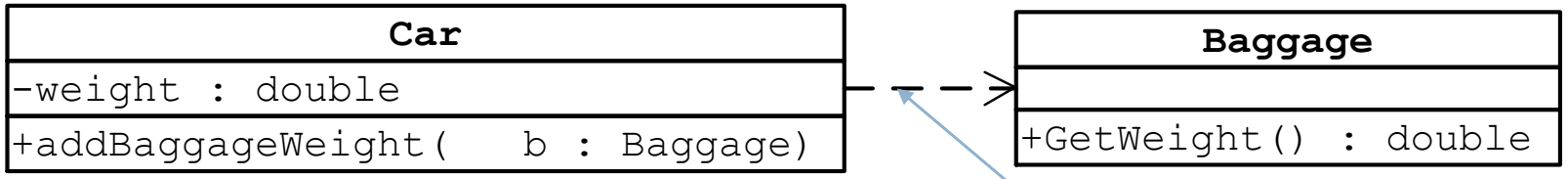


Dependency程式範例1




```
class X {
     …
     void f1(Y y) {…; y.foo(); }
     void f2() {Y y; y.foo();… }
}
```

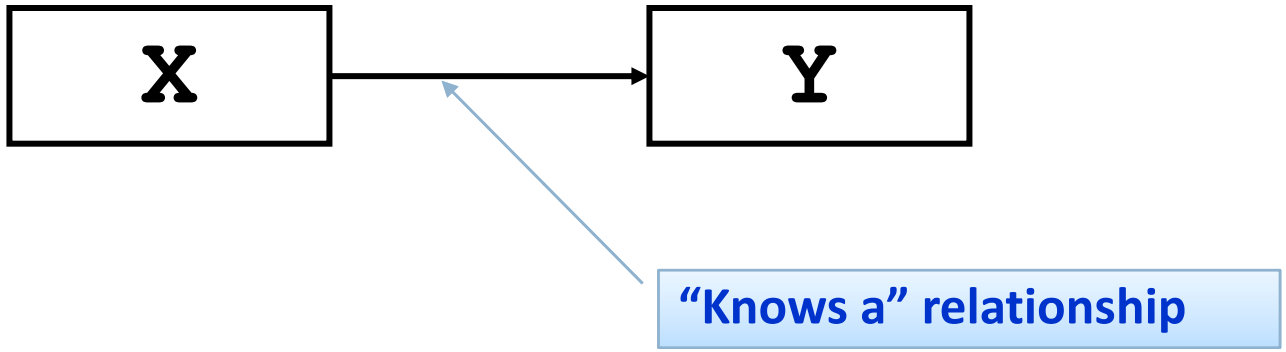
Dependency程式範例2




```
class Car {
    private double weight;
    …
    public void addBaggageWeight(Baggage baggage) {
        weight += baggage.GetWeight();
}
```

Car uses Baggage

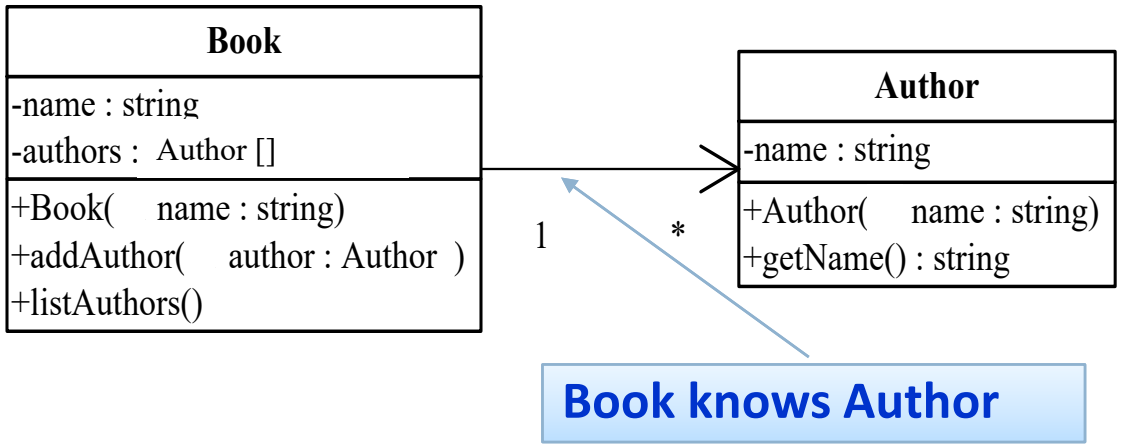
Association程式碼範例1




```
class X {
    private Y y; // X keeps a reference to Y

    public X(Y y) {
        this.y = y;
    }
}
```

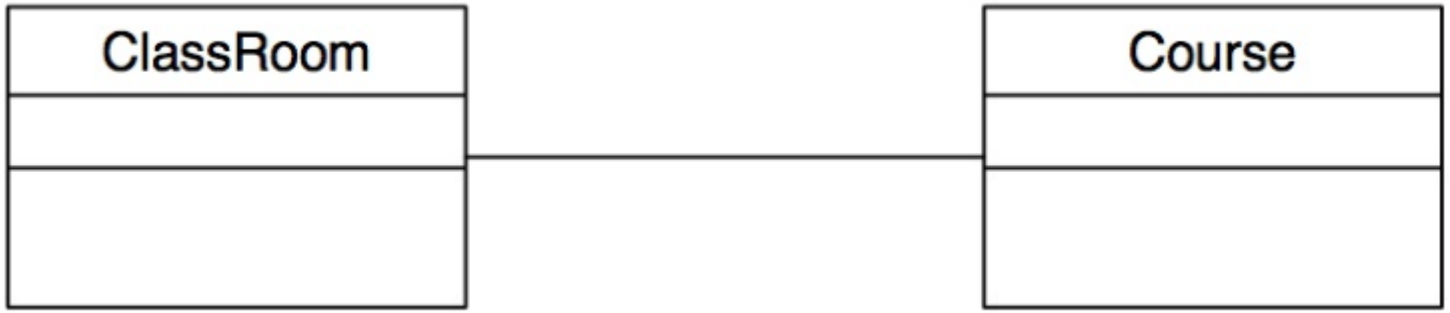
Association程式碼範例2




```
class Book {
    private string name;
    private Author[] authors;
    public Book(string name);
    void addAuthor(Author author) {…}
    void listAuthors() {
        for(int i = 0; i < authors.length; i++) {
            System.out.println(authors[i]);
    }
}
```

Navigation1

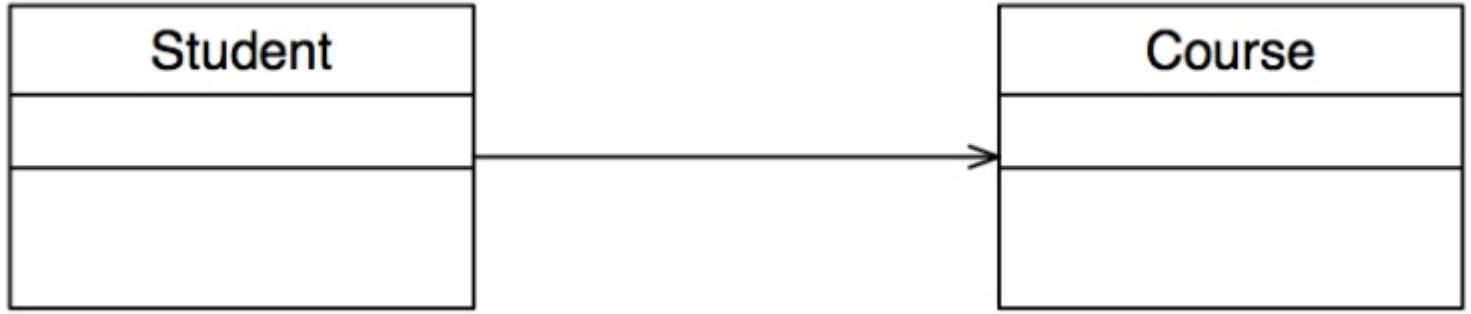
- 當兩邊的類別彼此都可看到對方時，我們可以用沒有箭頭的關聯線將兩端連接起來。




```
class ClassRoom {
    Course course;
}
class Course {
    ClassRoom room;
}
```

Navigation2

- 當Association關係中只有一方看得到另外一方，我們則用有箭頭的關聯線表達這樣的可視性(navigation)。

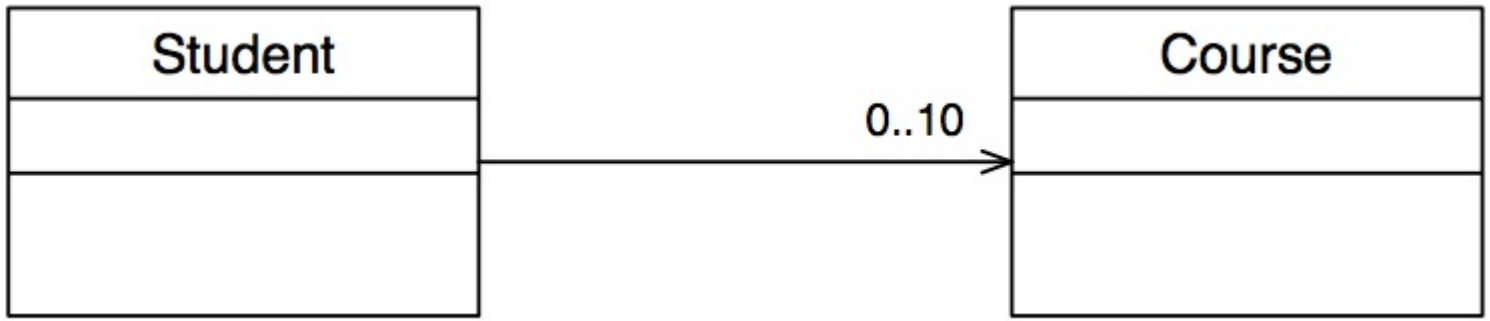



```
class Student {
      Course course;
}

class Course {
}
```

Multiplicity (數量)

- 關聯端點上可寫上數量，代表此參與此關聯關係之物件個數。

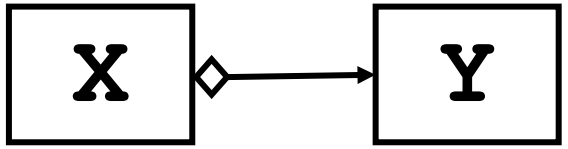


(一個學生可以上0到10門課)

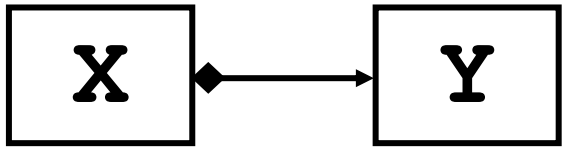
```
class Student {
    Course[] courses = new Course[10];
}
class Course {
}
```

Aggregation ⇔ Composition

- 組合(composition)和聚合(aggregation)都是 “Has a” 或“part-of”之關係
- 聚合(aggregation)
  - 在聚合關係中，若Y包含於X，Y可再被其他物件包含。
  - 當大物件X消失，小物件Y仍然存在。

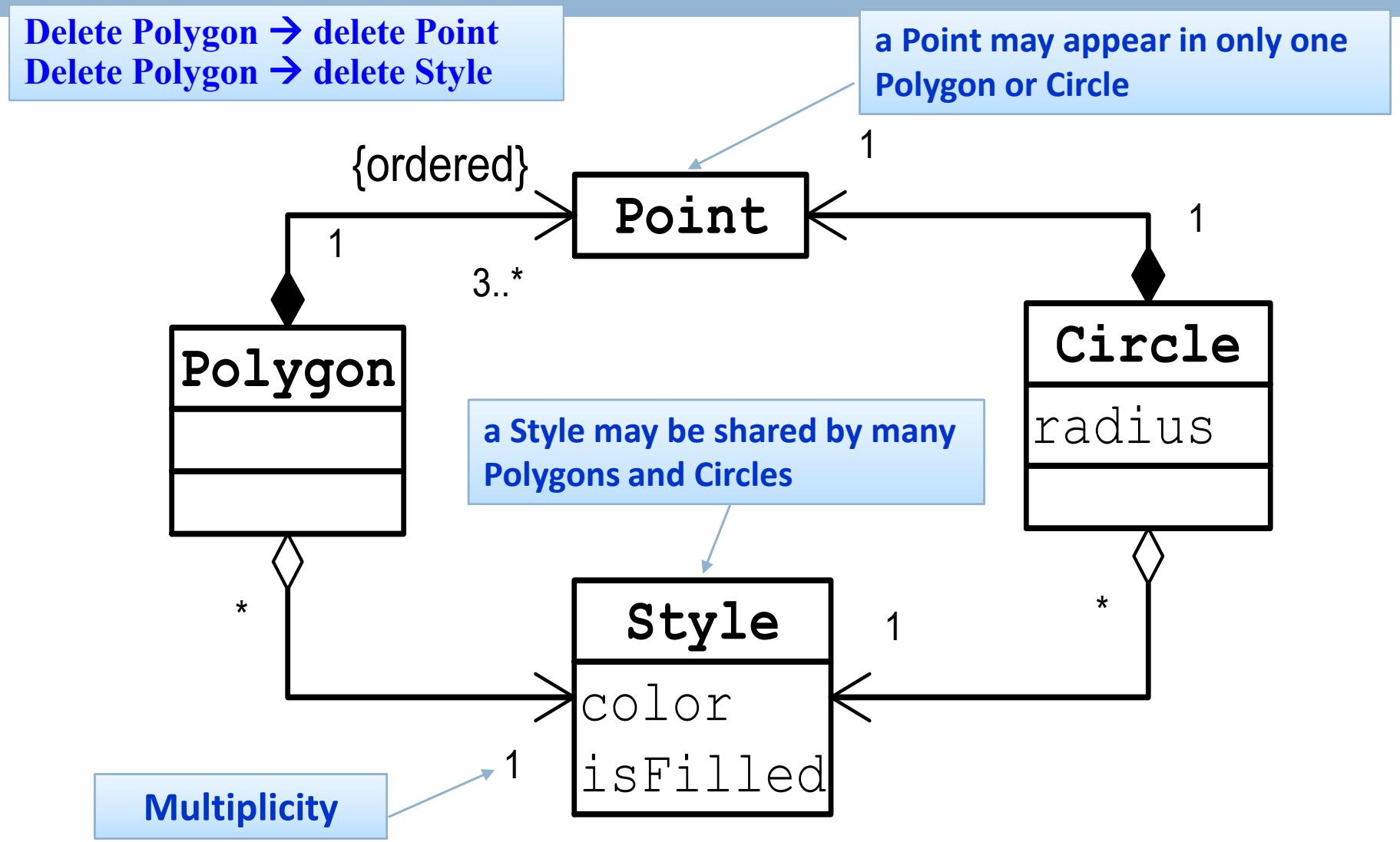


- 組合 (composition)
  - 包含的關係比聚合更為強烈
  - 在組合關係中，若Y包含於X，則Y不可被其他物件包含
  - X與Y之生命週期一致：若刪除X，則Y隨之被刪除。
  - 也被翻譯成合成

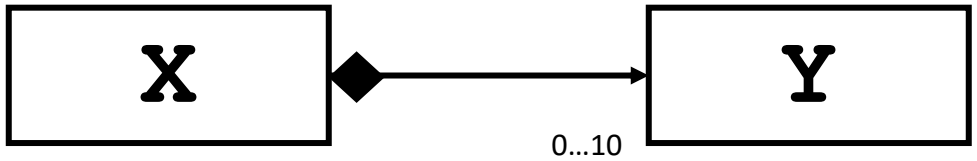


- 一些文獻會將這兩個概念都統稱為組合/合成。

範例: “has a” 關係

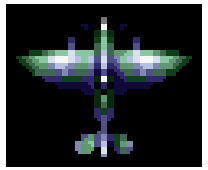

Delete Polygon → delete Point
Delete Polygon → delete Style
a Point may appear in only one Polygon or Circle
{ordered}
1
1
3..*
1
Point
Polygon
Circle
radius
a Style may be shared by many Polygons and Circles
*
Style
1
*
color
isFilled
1
Multiplicity

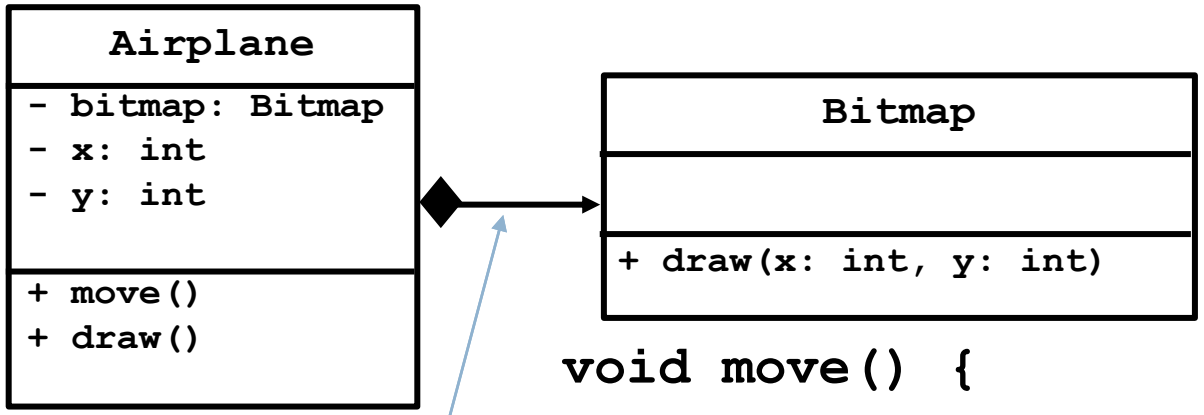
Composition程式範例1




```
class X {
    Y[] y;    // 0..10; Composition
    public X() { y = new Y[10]; }
    // protected void finalize(){ y = null; }
    …
}
```

Composition程式範例2






```
class Airplane {
    private Bitmap bitmap;
    private int x, y;

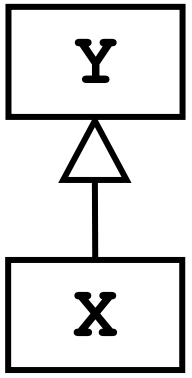
    void move() {
        x += …;
        y += …;
    }

    void draw() {
        bitmap.draw(x, y);
    }
}
```

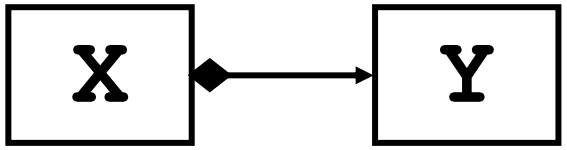
Airplane has Bitmap

“Is a” ⇔ “Has a” 關係

- “Is a” 關係
  - 繼承(inheritance)
  - X class is derived from Y

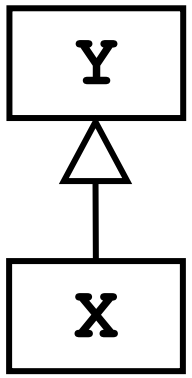


- “Has a”關係
  - 組合(composition)
  - X contains Y as members

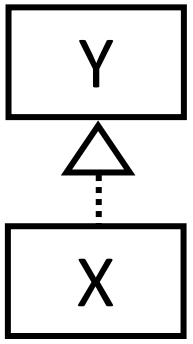


“Is a” ⇔ “Can do” relationship

- “Is a” 或”Is a kind of”關係
  - 繼承(inheritance)
  - X is derived from Y



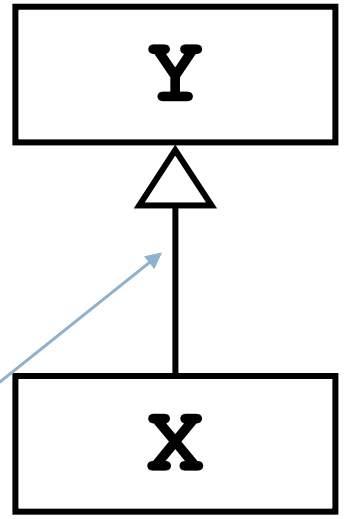
- “Can do” 關係
  - 實作(implementation)
  - X can do (replace) the jobs of Y



Inheritance程式範例


```
class Y {
    ...
}

class X extends Y {
    ...
}
```



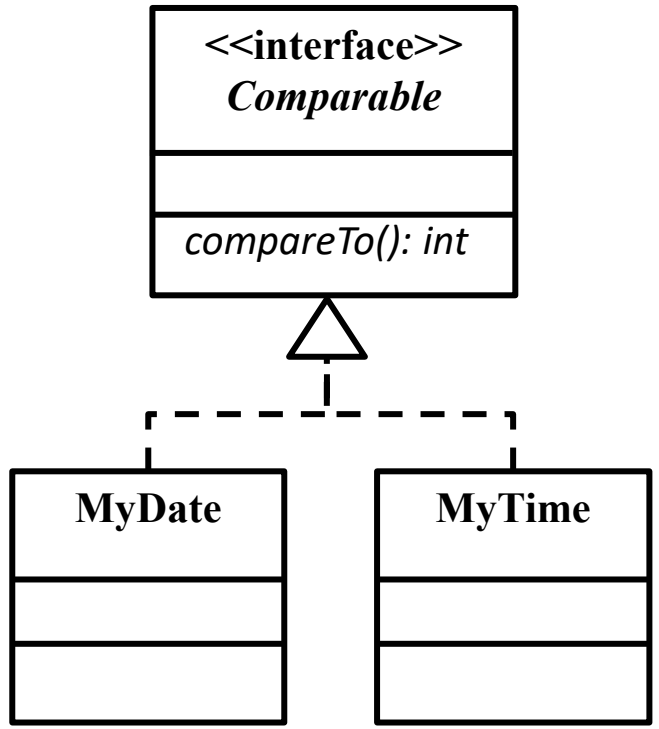
“is a” relationship

Implementation程式範例


```
interface Comparable {
    int compareTo(Object obj)
}

class MyDate implements Comparable {
    int compareTo(Object obj) {...}
    ...
}

class MyTime implements Comparable {
    int compareTo(Object obj) {...}
    ...
}
```



請回答下列物件間的關係

- Car ⇔ Engine ?
- Car ⇔ Baggage ?
- Driver ⇔ Car ?
- Driver ⇔ Baggage ?
- Person ⇔ Cell Phone ?
- Human ⇔ Brain ?
- Fighter ⇔ Bomb ?
- Fighter ⇔ F16
- Bomb ⇔ Nuclear Bomb (核子彈)
- Tree Node ⇔ Child of Tree Node ?
- Tree Node ⇔ Parent of Tree Node ?
- Hero ⇔ Life ? (Game)
- Hero ⇔ Map ? (Game)

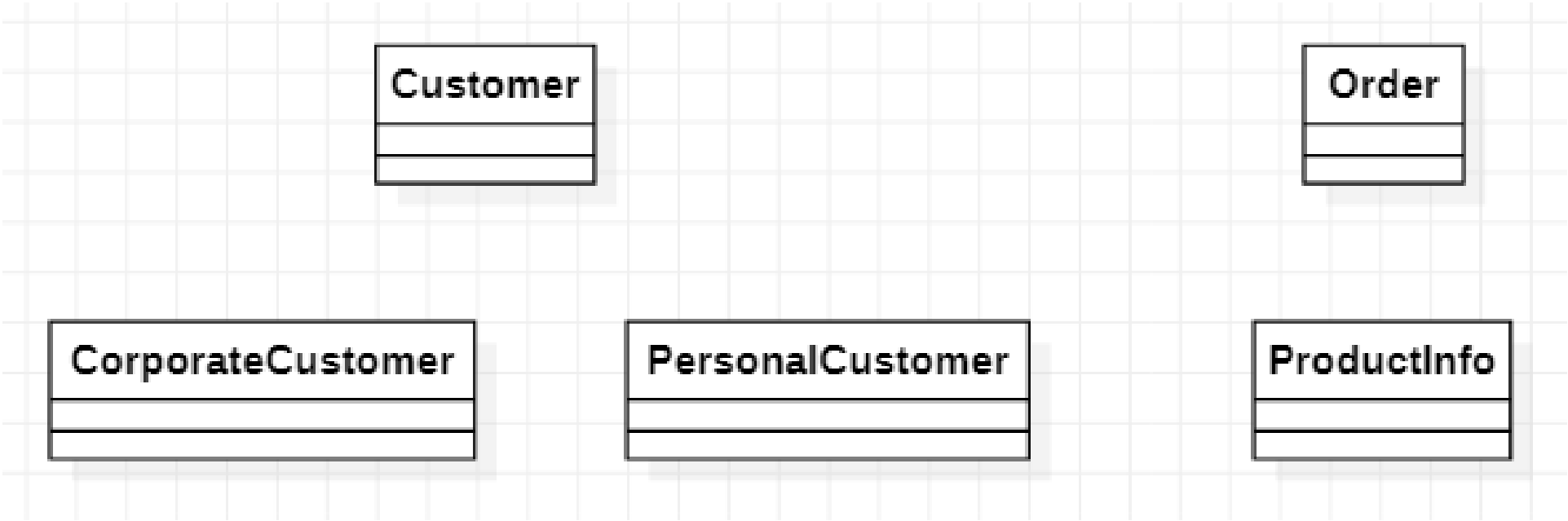
類別圖設計
(Class Diagram Design)

需求敘述(範例)

- 客戶有兩種，一類是公司客戶，另外一類為個人客戶。
- 客戶有名稱、住址等屬性資料。
- 個人客戶則額外有信用卡帳號資料。
- 公司客戶額外有信用狀態資料。
- 客戶會下訂單買產品。
- 產品需記錄編號、名稱、價錢等資料。

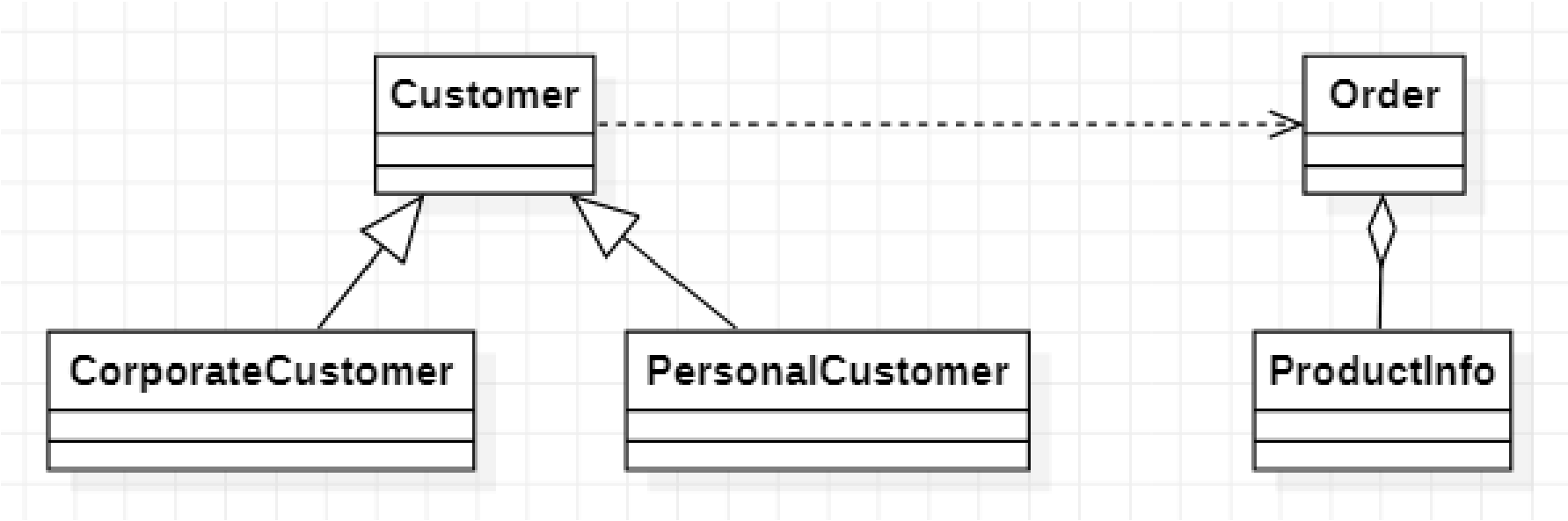
識別出所有的類別(Class)

- 找出可作為類別的名詞：
  - 客戶、公司客戶、個人客戶、訂單、產品資訊



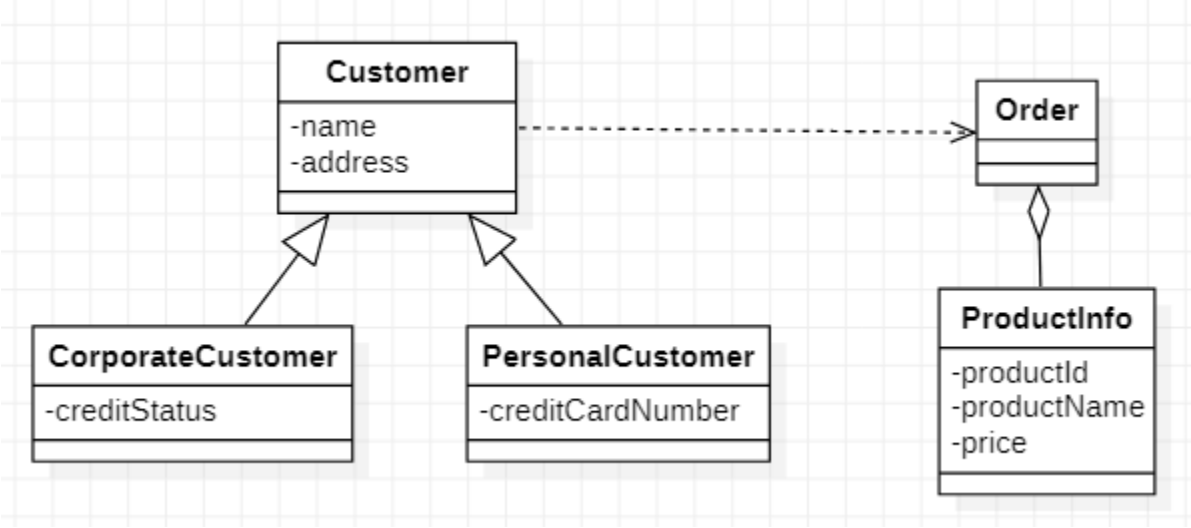
建立類別間的關係

- 客戶有兩種，一類是公司客戶，另外一類為個人客戶。
- 客戶會下訂單買產品。



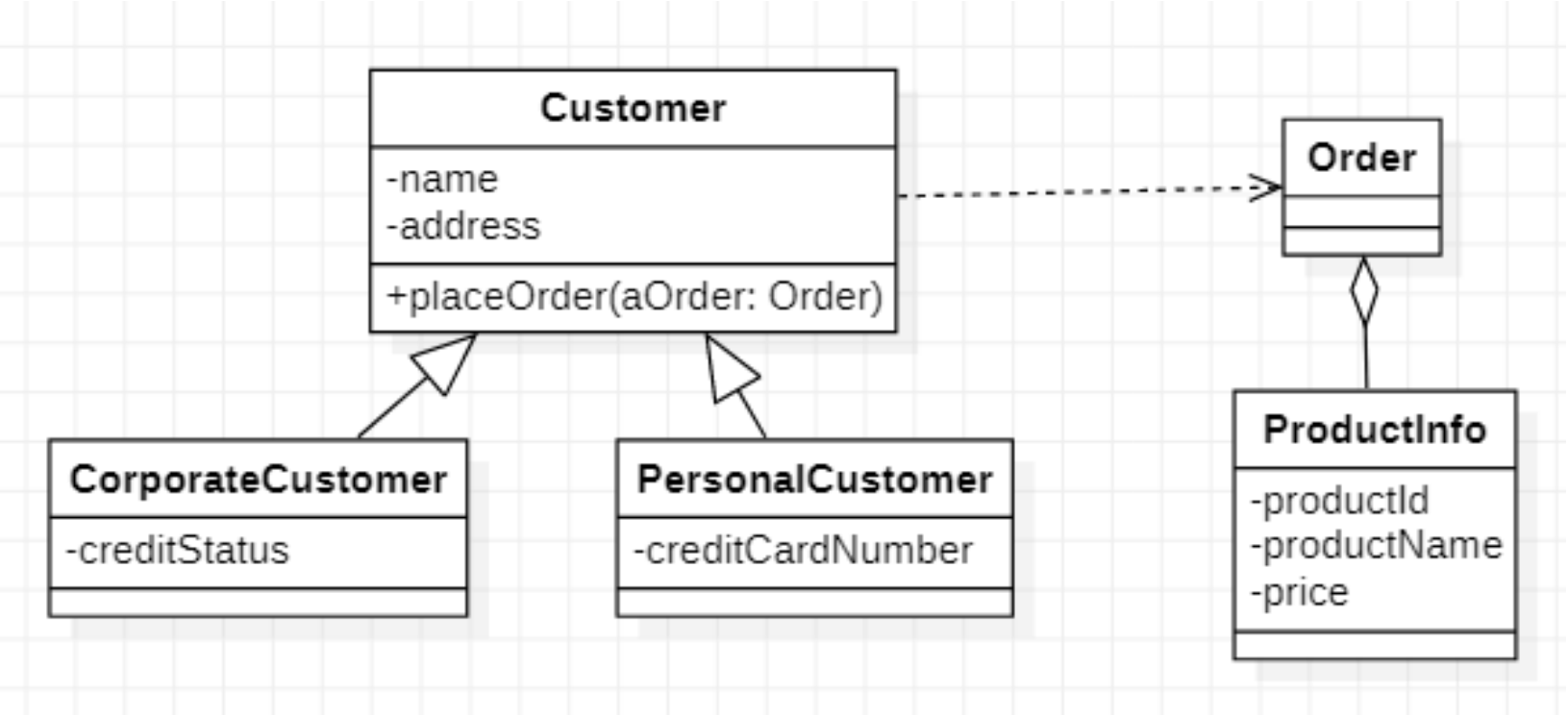
設定屬性(Attribute)

- 客戶有名稱、住址等屬性資料。
- 個人客戶則額外有信用卡帳號資料。
- 公司客戶額外有信用狀態資料。
- 產品需記錄編號、名稱、價錢等資料。



設定方法(Method)

- 客戶會下訂單買產品。

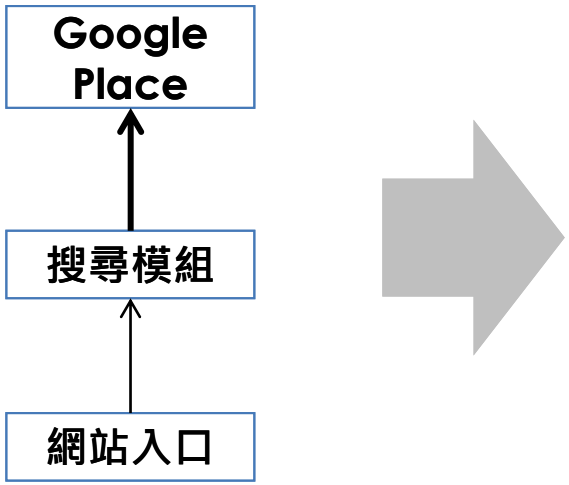


此為領域模型(Domain Model)，通常後續會再擴增，以建立更接近實作的類別圖。

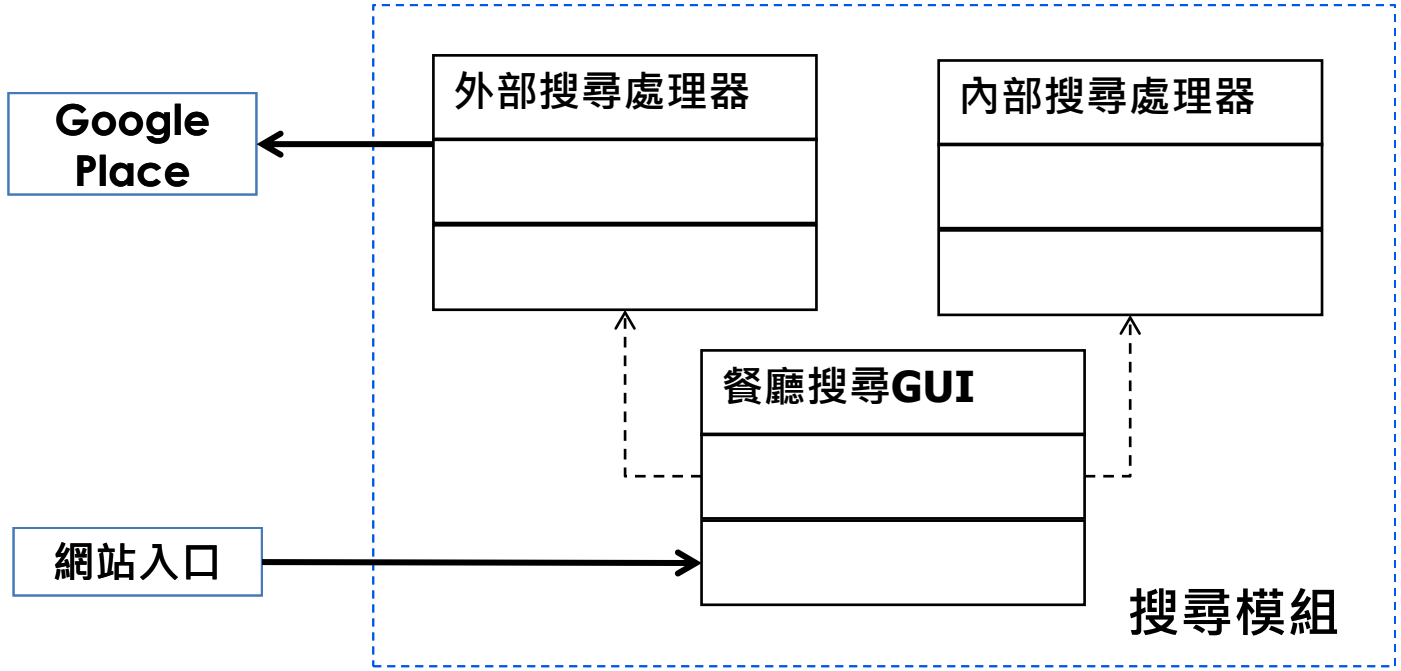
架構圖與類別圖

- 類別圖設計需與上層架構圖一致：
  - 上層架構圖中的介面，到了類別圖應盡量維持

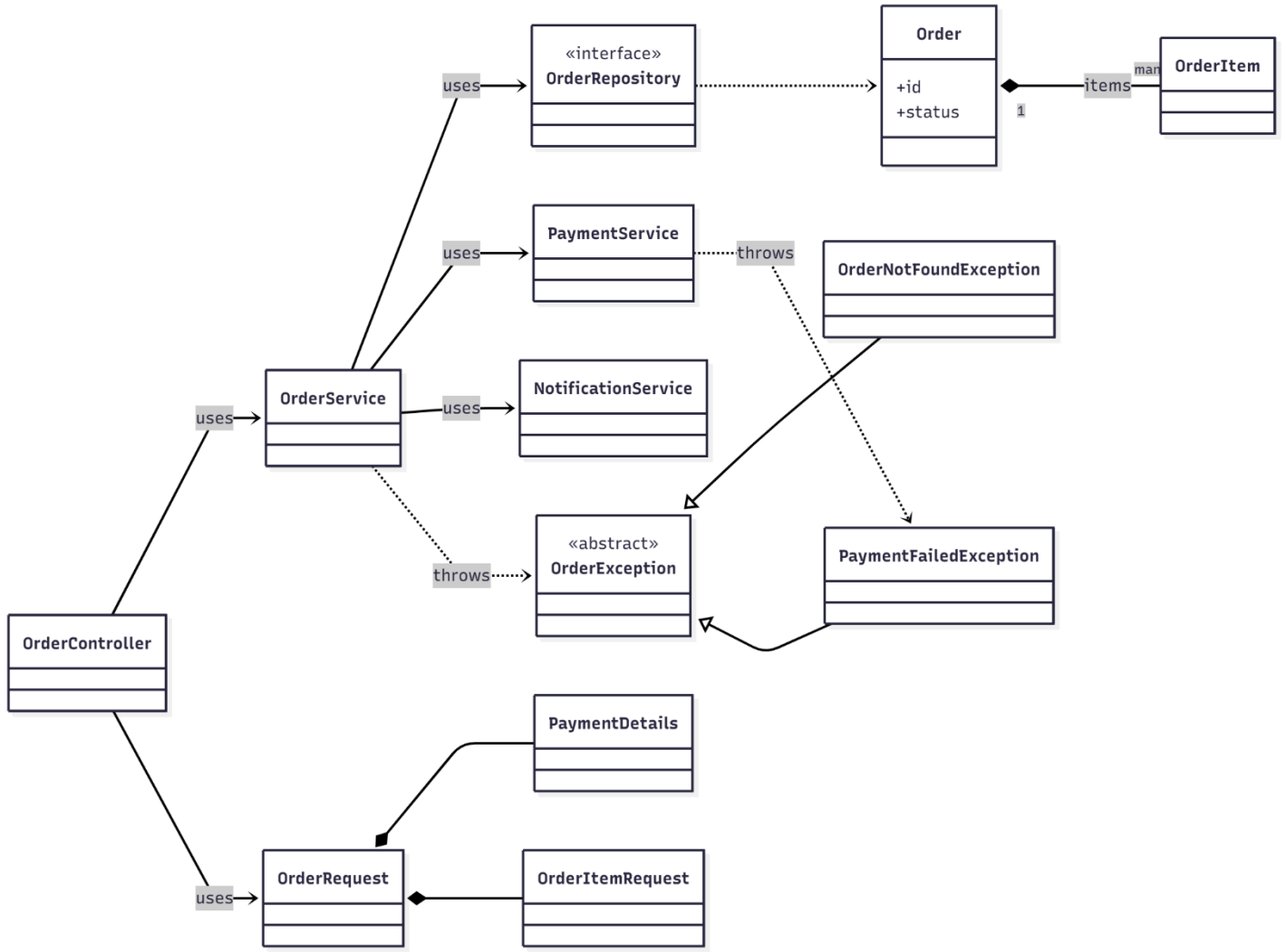
Component Architecture:



Code Architecture:

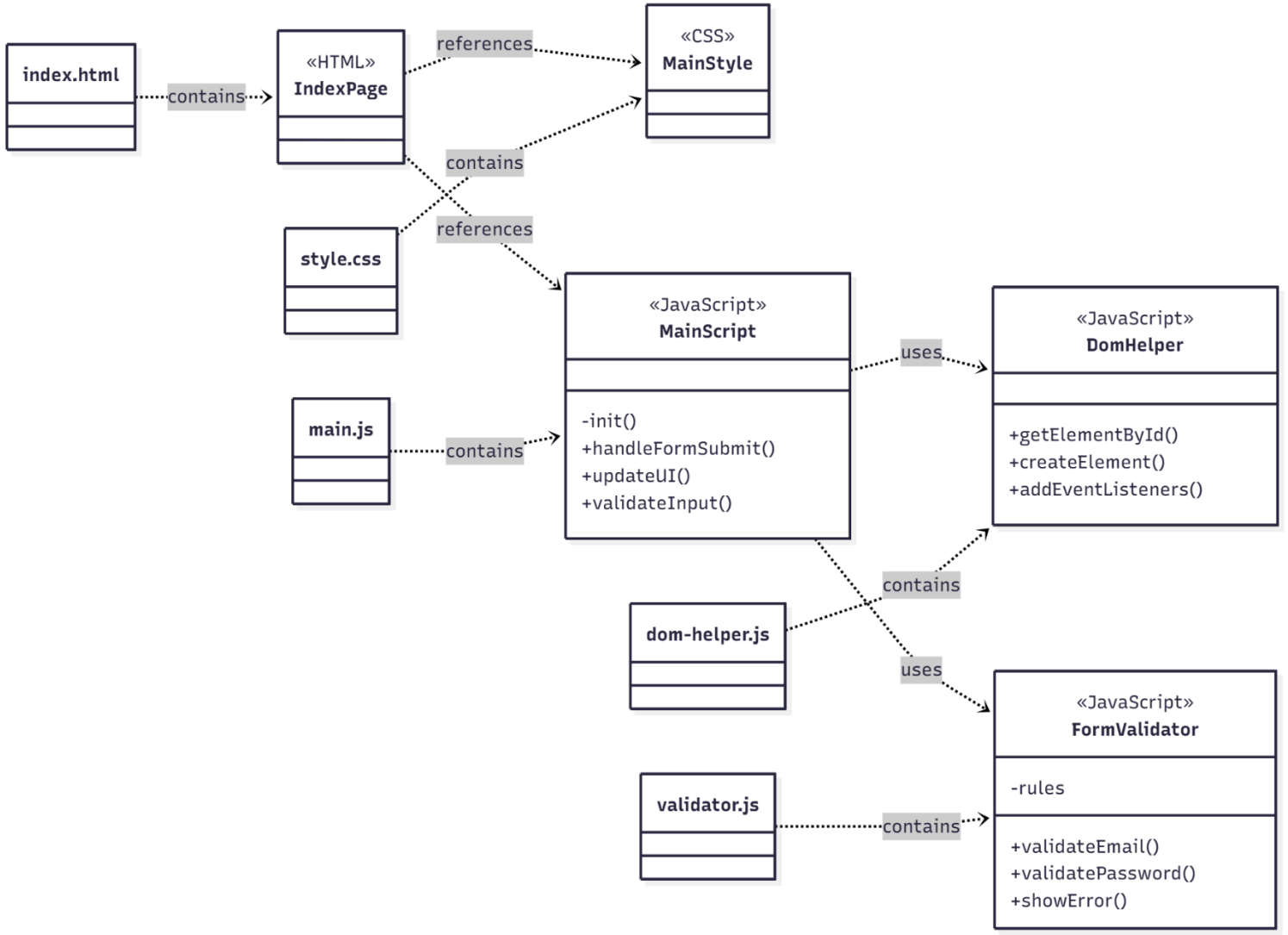


類別圖較完整案例



可能跨越多個Component

可運用Stereotype來呈現非OO之程式單元



Any Question?

